# Prompting Claude Fable 5

Behavioral differences and prompting patterns for Claude Fable 5 and Claude Mythos 5, covering effort, instruction following, long runs, memory, and scaffolding changes.

Copy page

This guide covers the prompting and scaffolding patterns specific to Claude Fable 5 and Claude Mythos 5. For the model's capabilities, API changes, pricing, and availability, see Introducing Claude Fable 5 and Claude Mythos 5. For techniques that apply across all current Claude models, see Prompting best practices.

Claude Fable 5 takes on problems that were previously too complex, long-running, or ambiguous for prior models, and is particularly effective at end-to-end work that takes a person hours, days, or weeks to complete. The teams seeing the best outcomes apply Claude Fable 5 to their hardest unsolved problems; testing it only on simpler workloads tends to undersell its capability range. It also performs reliably on more straightforward tasks.

Claude Fable 5 has several behavioral differences from Claude Opus 4.8 that may require prompt or scaffolding updates. Capability improvements at this level are also a good prompt to re-evaluate which instructions, tools, and guardrails are still needed. The patterns below cover the behaviors that most often require tuning.

For API parameter changes specific to Claude Fable 5 and Claude Mythos 5 (adaptive thinking only, summarized-only thinking output, no extended thinking budgets, the `refusal` stop reason and fallback handling), see Introducing Claude Fable 5 and Claude Mythos 5.

Claude Fable 5 runs safety classifiers that target offensive cybersecurity techniques (such as building exploits, malware, or attack tooling), biology and life sciences content (such as lab methods or molecular mechanisms), and extraction of the model's summarized thinking. Benign cybersecurity work and beneficial life sciences tasks may also trigger these safeguards. To re-route declined requests automatically, configure server-side or client-side fallback to Claude Opus 4.8.

## Capability improvements

Compared with Claude Opus 4.8, Claude Fable 5 shows improvement in:

- **Long-horizon autonomy.** Claude Fable 5 sustains productive output over extended periods, completing multi-day, goal-directed runs with strong instruction retention across long, complex tasks.
- **First-shot correctness on complex, well-specified problems.** Early testers reported single-pass implementations of systems that previously took days of iteration.
- **Vision.** Claude Fable 5 interprets dense technical images, web applications, and detailed screenshots with substantially higher accuracy, often while using fewer output tokens, and is trained to use bash and crop tools to handle flipped, blurry, or noisy images.
- **Enterprise workflows.** Claude Fable 5 follows instructions, stays in scope, and produces output on financial analysis, spreadsheets, slides, and documents.

- **Delegation and collaboration.** Claude Fable 5 is significantly more dependable at dispatching and sustaining parallel subagents, and reliably manages ongoing communication with long-running subagents and peer agents.

Beyond these specific improvements, Claude Fable 5 is generally more capable than prior models on almost all tasks. Claude Fable 5 is not intended for offensive cybersecurity or biology and life sciences work; requests in those domains can return `stop_reason: "refusal"`.

## Longer turns by default

Individual requests on hard tasks can run for many minutes at higher effort settings, especially when the task requires gathering context, building, and self-verifying, and autonomous runs can extend for hours. This is one of the largest shifts teams encounter when adjusting to Claude Fable 5. Adjust client timeouts, streaming, and user-facing progress indicators before migrating, and consider restructuring harnesses to check on runs asynchronously, for example through scheduled jobs, rather than blocking. To keep Claude Fable 5 from overplanning when a task is ambiguous:

```
When you have enough information to act, act. Do not re-derive facts already
established in the conversation, re-litigate a decision the user has already made,
or narrate options you will not pursue in user-facing messages. If you are weighing
a choice, give a recommendation, not an exhaustive survey. This does not apply to
thinking blocks.
```

## Consider all effort levels

Effort is the primary control for the trade-off between intelligence, latency, and cost on Claude Fable 5. Use `high` as the default for most tasks, with `xhigh` for the most capability-sensitive workloads and `medium` or `low` for routine work. Lower effort settings on Claude Fable 5 still perform well and often exceed `xhigh` performance on prior models. Reduce effort if a task completes but takes longer than necessary, or if you want a quicker, more interactive working style.

On routine work at higher effort, Claude Fable 5 can gather context and deliberate beyond what the task needs. At the same time, higher effort often produces excellent verification behavior, sophisticated reasoning, and the most rigorous output. To prevent unrequested tidying or refactoring at higher effort:

```
Don't add features, refactor, or introduce abstractions beyond what the task
requires. A bug fix doesn't need surrounding cleanup and a one-shot operation
usually doesn't need a helper. Don't design for hypothetical future requirements: do
the simplest thing that works well. Avoid premature abstraction and half-finished
implementations. Don't add error handling, fallbacks, or validation for scenarios
that cannot happen. Trust internal code and framework guarantees. Only validate at
system boundaries (user input, external APIs). Don't use feature flags or
backwards-compatibility shims when you can just change the code.
```

enumerating each behavior by name. For example, when un-steered, Claude Fable 5 can elaborate beyond what the task needs, especially at higher effort settings: surveying options it won't pursue, explaining root causes at length, producing heavily-structured PR descriptions, or writing comments that narrate what the next line does. A short brevity instruction is as effective as listing each pattern:

```
Lead with the outcome. Your first sentence after finishing should answer "what happened" or "what did you find": the thing the user would ask for if they said "just give me the TLDR." Supporting detail and reasoning come after. Being readable and being concise are different things, and readability matters more.

The way to keep output short is to be selective about what you include (drop details that don't change what the reader would do next), not to compress the writing into fragments, abbreviations, arrow chains like A → B → fails, or jargon.
```

The same applies to checkpoint behavior in long-running workflows. To have Claude Fable 5 stop only where it genuinely needs you, there is no need to enumerate every case:

```
Pause for the user only when the work genuinely requires them: a destructive or irreversible action, a real scope change, or input that only they can provide. If you hit one of these, ask and end the turn, rather than ending on a promise.
```

## Ground progress claims during long runs

On long autonomous runs, instruct Claude Fable 5 to audit progress against actual tool results. In Anthropic's testing, this nearly eliminated fabricated status reports even on tasks designed to elicit them:

```
Before reporting progress, audit each claim against a tool result from this session. Only report work you can point to evidence for; if something is not yet verified, say so explicitly. Report outcomes faithfully: if tests fail, say so with the output; if a step was skipped, say that; when something is done and verified, state it plainly without hedging.
```

## State the boundaries

Claude Fable 5 can occasionally take unrequested actions (drafting an email when none was asked for, creating defensive git-branch backups). Define explicit constraints on what Claude Fable 5 should and should not do:

```
evidence actually supports that specific action. A signal that pattern-matches to a
known failure may have a different cause.
```

## Parallel subagents

Claude Fable 5 dispatches parallel subagents more readily than prior models. Use subagents frequently, provide explicit guidance about when delegation is appropriate, and prefer asynchronous communication between orchestrator and subagents over blocking until each subagent returns. Long-lived subagents that keep their context across subtasks save time and cost through cache reads and avoid bottlenecking on the slowest subagent.

```
Delegate independent subtasks to subagents and keep working while they run.
Intervene if a subagent goes off track or is missing relevant context.
```

## Construct a memory system

Claude Fable 5 performs particularly well when it can record lessons from previous runs and reference them. Provide a place to write notes, as simple as a Markdown file:

```
Store one lesson per file with a one-line summary at the top. Record corrections and
confirmed approaches alike, including why they mattered. Don't save what the repo or
chat history already records; update an existing note rather than creating a
duplicate; delete notes that turn out to be wrong.
```

To bootstrap the memory system from existing history, have Claude Fable 5 review past sessions:

```
Reflect on the previous sessions we've had together. Use subagents to identify core
themes and lessons, and store them in [X]. Make sure you know to reference [X] for
future use.
```

## Rare cases of early stopping

Deep into a long session, Claude Fable 5 can occasionally end a turn with a text-only statement of intent ("I'll now run X") without issuing the corresponding tool call, or pause to ask permission when it already has enough to proceed. A "continue" or "go ahead and do it end to end" suffices. To define when pausing is appropriate, pair this with the checkpoint instruction in Strong instruction following. For autonomous pipelines, add a system reminder:

```
already discussing with the user before doing the work is not. Before ending your
turn, check your last paragraph. If it is a plan, an analysis, a question, a list of
next steps, or a promise about work you have not done ("I'll…", "let me know
when…"), do that work now with tool calls. End your turn only when the task is
complete or you are blocked on input only the user can provide.
```

## Rare cases of context-budget concern

In very long sessions, Claude Fable 5 can occasionally suggest a new session, offer to summarize and hand off, or trim its own work. This is most often triggered when the harness shows a remaining-token countdown to the model. Avoid surfacing explicit context-budget counts where possible. If the harness must show them, a reassurance helps:

```
You have ample context remaining. Do not stop, summarize, or suggest a new session
on account of context limits. Continue the work.
```

## Give the reason, not only the request

Claude Fable 5 tends to perform better when it understands the intent behind a request: context lets it connect the task to relevant information rather than inferring intent on its own. Provide context about why you're asking, especially for long-running agents drawing on multiple workstreams:

```
I'm working on [the larger task] for [who it's for]. They need [what the output
enables]. With that in mind: [request].
```

## Readability when communicating with the user

In extended or agentic conversations (many tool calls, large working context), Claude Fable 5 can produce text that's hard to follow: dense arrow-chain shorthand, deep implementation detail, references to thinking the user never saw, or overly technical phrasing. A communication-style addendum mitigates this:

```
Terse shorthand is fine between tool calls (that's you thinking out loud, and
brevity there is good). Your final summary is different: it's for a reader who
didn't see any of that.
```

```
unless you re-introduce it.

When you write the summary at the end, drop the working shorthand. Write complete
sentences. Spell out terms. Don't use arrow chains, hyphen-stacked compounds, or
labels you made up earlier. When you mention files, commits, flags, or other
identifiers, give each one its own plain-language clause. Open with the outcome: one
sentence on what happened or what you found. Then the supporting detail. If you have
to choose between short and clear, choose clear.
```

## Create a send-to-user tool

When running long, asynchronous agents, give the agent a way to surface a message the user must see exactly as written, without ending its turn: a deliverable (a generated code snippet or a drafted message), a progress update with specific numbers, or a direct reply to a question the user asked mid-loop. The tool's input is the message to display; when Claude calls it, render the input directly in your UI and return a simple acknowledgement as the tool result. Tool inputs are never summarized, so the content arrives intact.

```
{
  "name": "send_to_user",
  "description": "Display a message directly to the user. Use this for progress updates
  "input_schema": {
    "type": "object",
    "properties": {
      "message": {
        "type": "string",
        "description": "The content to display to the user."
      }
    },
    "required": ["message"]
  }
}
```

Add this tool whenever your UX depends on delivering content or direct user interactions verbatim mid-task. For agents that only narrate routine progress, the model's own summaries are typically adequate. Defining the tool is not sufficient on its own; without an instruction in the system prompt, Claude Fable 5 rarely calls it. Pair the tool with elicitation language such as:

```
Between tool calls, when you have content the user must read verbatim (a partial
deliverable, a direct answer to their question), call the send_to_user tool with
that content. Use send_to_user only for user-facing content, not for narration or
reasoning.
```

- **Start at the top of your difficulty range.** Pick a task harder than what you'd assign to prior models, and have Claude Fable 5 scope it, ask clarifying questions, and execute.
- **Make self-verification explicit in long-run prompts.** Separate, fresh-context verifier subagents tend to outperform self-critique. For long-running tasks, instruct: `Establish a method for checking your own work at an interval of [X] as you build. Run this every [X interval], verifying your work with subagents against the specification.`
- **Refactor existing prompts and skills.** Skills developed for prior models are often too prescriptive for Claude Fable 5 and can degrade output quality. Review and consider removing older instructions if default performance is better. Claude Fable 5 also does a good job of updating skills on the fly based on what it learns from the task at hand.
- **Don't instruct Claude to reproduce its reasoning in the response.** Prompts, skills, or harness instructions that tell the model to echo, transcribe, or explain its internal reasoning as response text can trigger the `reasoning_extraction` refusal category on Claude Fable 5, causing elevated fallbacks to Claude Opus 4.8. Audit existing skills and system prompts for reflection or show-your-thinking instructions when migrating. If your application needs reasoning visibility, read the structured `thinking` blocks from adaptive thinking instead, and use a send-to-user tool to surface progress during long runs.
- **Create a send-to-user tool.** For long, asynchronous agents, a client-side tool delivers messages to the user verbatim without ending the turn. See Create a send-to-user tool.